



Evolutionary Computation

Assignment 5: The use of move evaluations (deltas) from previous iterations in local search

BENGRICHE Tidiane (ER-2037)
SZCZUBLIŃSKA Joanna Wiktoria (156070)



Contents

1	Introduction	1
2	Pseudocode	2
3	Plots	7
4	Results	9
5	Conclusion	9
6	All_Results	10

1 Introduction

The purpose of this report is to improve the time efficiency of the steepest local search algorithm by reusing move evaluations from previous iterations through a list of improving moves (LM). While steepest local search provides high-quality solutions, its main drawback is the computational cost of re-evaluating the entire neighborhood at every iteration. By maintaining a list of previously discovered improving moves and checking their applicability before performing a full neighborhood scan, we aim to significantly reduce the number of delta calculations.

The list of moves stores both intra-route and inter-route (node exchange) moves discovered during the search. For intra-route moves, we record the four nodes defining the two removed edges along with their relative direction. A key feature is the handling of edge directions when browsing the LM:

- Removed edges no longer exist $\rightarrow$ remove the move from LM
- Removed edges occur in a different relative direction $\rightarrow$ leave in LM but skip (may be applicable later)
- Removed edges appear in the same relative direction (or both reversed) $\rightarrow$ re-evaluate and apply if still improving

Additionally, when evaluating new moves, we consider both current and inverted edge directions to maximize delta reuse across iterations.

Random solutions are used as starting points. For comparison, we report results of standard steepest local search without the LM mechanism as a baseline, allowing us to measure improvements in computational time and any impact on solution quality.

You can see the repository of our assignment on github:

https://github.com/tbengric/EC_laboratory

2 Pseudocode

In this section, we will present the pseudocodes of the algorithm. For this assignment, we will code in C++.

Steepest Local Search with LM (part 1)

```
FUNCTION steepest_local_search_with_lm(initial_path, dist_matrix,
nodes, list_of_moves):
  current_path ← COPY(initial_path)
  path_size ← LENGTH(current_path)
  n ← LENGTH(nodes)

  REPEAT :
    selected_nodes ← SET(current_path)
    not_selected_nodes ← {node ∈ nodes : node ∉ selected_nodes}
    lm_move_applied ← FALSE

    PHASE 1: Browse list of moves (LM)
    FOR move IN list_of_moves DO :
      IF move.type = 1 THEN : Intra-edge move
        dir1 ← edge_exists_and_direction(current_path,
move.removed_node1, move.removed_node2)
        dir2 ← edge_exists_and_direction(current_path,
move.removed_node3, move.removed_node4)

        IF dir1 = 0 OR dir2 = 0 THEN : Edge no longer exists
          REMOVE move FROM list_of_moves
          CONTINUE

        same_direction ← (dir1 = move.edge_dir1 AND dir2 =
move.edge_dir2)
        both_reversed ← (dir1 = -move.edge_dir1 AND dir2 =
-move.edge_dir2)

        IF same_direction OR both_reversed THEN : Applicable
          pos1 ← FIND_INDEX(current_path, move.removed_node1)
          pos3 ← FIND_INDEX(current_path, move.removed_node3)
          delta ← delta_intra_edge(current_path, pos1, pos3, dist_matrix)
          IF delta < 0 THEN :
            REVERSE(current_path, from: MIN(pos1,pos3)+1, to:
MAX(pos1,pos3)+1)
            REMOVE move FROM list_of_moves
            lm_move_applied ← TRUE
            BREAK
          ELSE : REMOVE move FROM list_of_moves
        ELSE : Different direction - keep in LM
          CONTINUE
```

Steepest Local Search with LM (part 2)

```
ELSE IF move.type = 0 OR move.type = 2 THEN : Node moves
  posA ← FIND_INDEX(current_path, move.saved_node_a)
  IF posA = -1 THEN : REMOVE move FROM list_of_moves;
CONTINUE

IF move.type = 0 THEN : Intra-node
  posB ← FIND_INDEX(current_path, move.saved_node_b)
  IF posB = -1 THEN : REMOVE move FROM list_of_moves;
CONTINUE
  delta ← delta_intra_node(current_path, posA, posB, dist_matrix)
  IF delta < 0 THEN :
    SWAP(current_path[posA], current_path[posB])
    REMOVE move FROM list_of_moves
    lm_move_applied ← TRUE
    BREAK

ELSE IF move.type = 2 THEN : Inter-node
  IF move.third_param ∈ selected_nodes THEN : REMOVE move
FROM list_of_moves; CONTINUE
  delta ← delta_inter_node(current_path, posA, move.third_param,
dist_matrix, nodes)
  IF delta < 0 THEN :
    current_path[posA] ← move.third_param
    REMOVE move FROM list_of_moves
    lm_move_applied ← TRUE
    BREAK

IF lm_move_applied THEN : CONTINUE Go to next iteration

PHASE 2: Evaluate neighborhood and collect improving moves
best_delta ← 0
best_move_type ← -1
best_first, best_second, best_not_selected ← -1
new_improving_moves ← []
```

Steepest Local Search with LM (part 3)

Intra-route moves

```
FOR i FROM 0 TO path_size-1 DO :  
  FOR j FROM i+2 TO path_size DO :  
    delta ← delta_intra_edge(current_path, i, j, dist_matrix)  
    IF delta < 0 THEN :  
      IF delta < best_delta THEN :  
        best_delta ← delta  
        best_move_type ← 1  
        best_first, best_second ← i, j
```

Store move with current edge directions

```
new_move ← CREATE_SAVED_MOVE(1, i, j, current_path)  
ADD new_move TO new_improving_moves
```

Also store move with inverted edge directions

```
inv_move ← COPY(new_move)  
inv_move.edge_dir1 ← -inv_move.edge_dir1  
inv_move.edge_dir2 ← -inv_move.edge_dir2  
ADD inv_move TO new_improving_moves
```

Inter-route moves

```
FOR i FROM 1 TO path_size-2 DO :  
  FOR not_selected_node IN not_selected_nodes DO :  
    delta ← delta_inter_node(current_path, i, not_selected_node,  
dist_matrix, nodes)  
    IF delta < 0 THEN :  
      IF delta < best_delta THEN :  
        best_delta ← delta  
        best_move_type ← 2  
        best_first ← i  
        best_not_selected ← not_selected_node  
  
        new_move ← CREATE_SAVED_MOVE(2, i, not_selected_node,  
current_path)  
        ADD new_move TO new_improving_moves
```

Add new improving moves to LM

```
SORT new_improving_moves BY delta (ascending)
```

```
FOR move IN new_improving_moves DO : ADD move TO list_of_moves
```

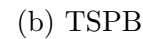
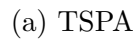
Steepest Local Search with LM (part 4)

```
Apply best move or terminate
IF best_move_type = 1 THEN :
    REVERSE(current_path, from: best_first+1, to: best_second+1)
    REMOVE applied move FROM list_of_moves
ELSE IF best_move_type = 2 THEN :
    current_path[best_first] ← best_not_selected
    REMOVE applied move FROM list_of_moves
ELSE : BREAK No improving move found

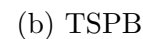
RETURN current_path
```

After trying this algorithm, we will check its effectiveness by showing the constructed cycle.

Figures for Steepest Local Search for Random Solution



(a) TSPA



7

Figures for Steepest Local Search with Candidate Moves

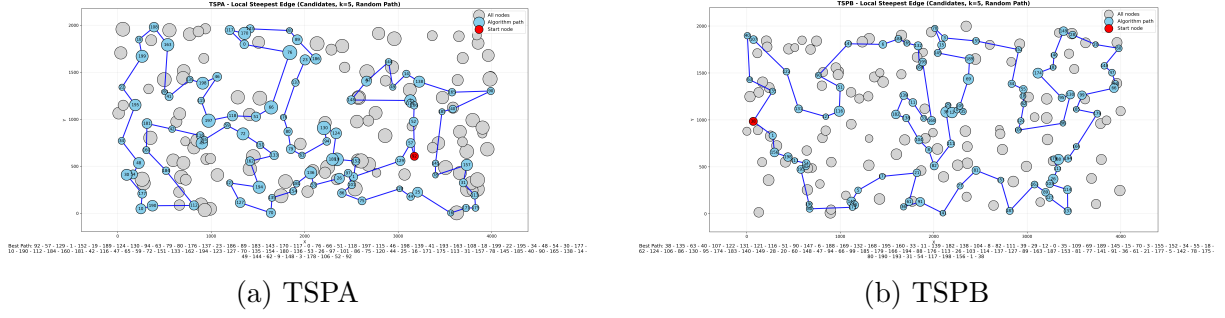


Figure 3: Local Steepest Node (Random Path)

Figures for Steepest Local Search with Candidate Moves

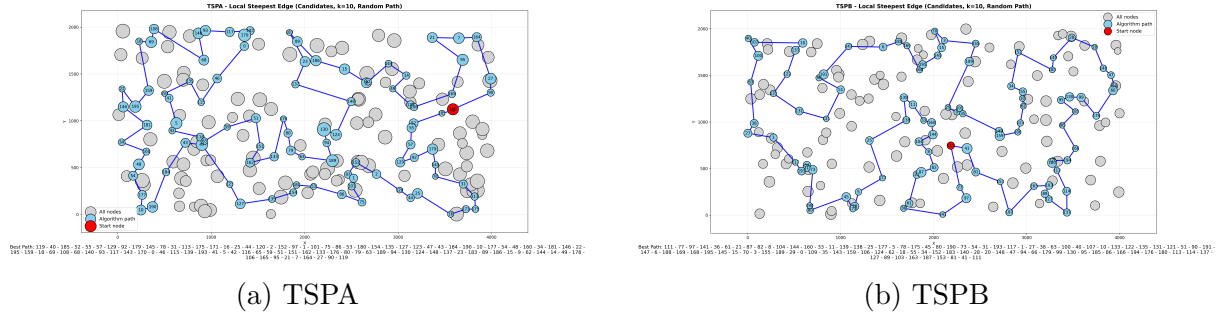


Figure 4: Local Steepest Node (Random Path)

Figures for Steepest Local Search with Candidate Moves

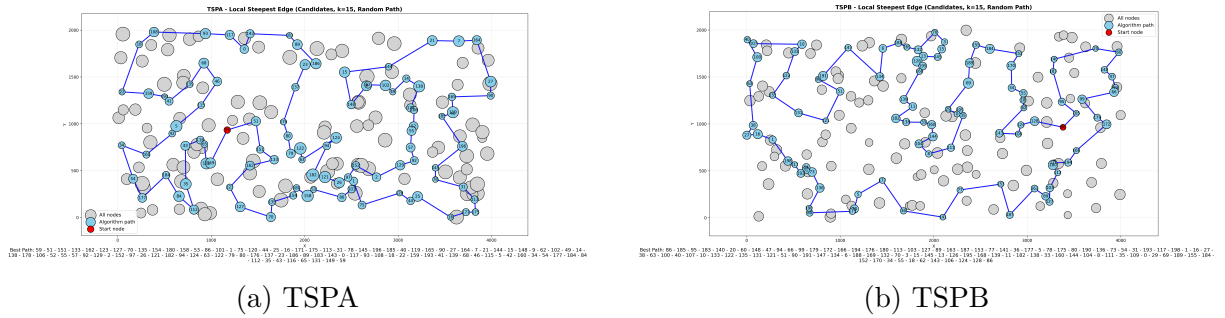


Figure 5: Local Steepest Node (Random Path)

Figures for Steepest Local Search with LM

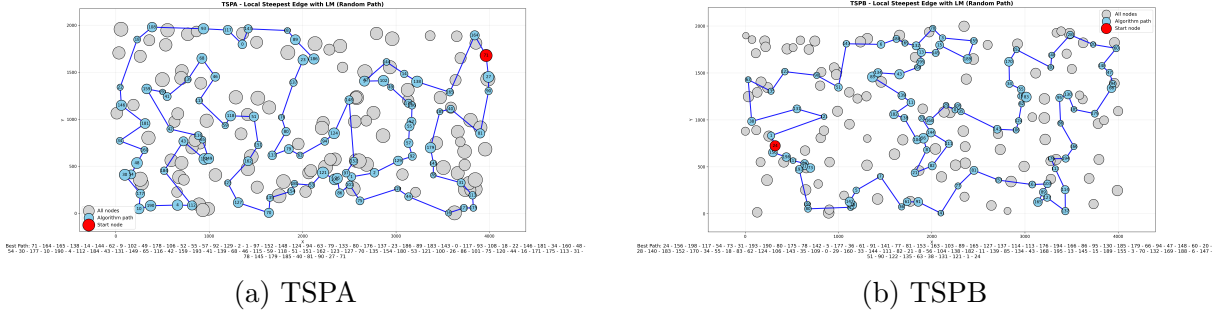


Figure 6: Local Steepest Edge with LM (Random Path)

4 Results

Method	TSPA (Avg, Min, Max)	TSPB (Avg, Min, Max)
Random Path	264441.33 (240555, 292576)	213493.24 (190998, 241645)
LS Edge (Random Path)	74072.76 (71504, 80317)	48460.08 (45888, 51408)
LS Edge with LM (Random Path)	74187.41 (71646, 78764)	49144.47 (45888, 53039)
LS Edge (Candidates, k=5)	86259.51 (81102, 92277)	50647.90 (47462, 55655)
LS Edge (Candidates, k=10)	78037.45 (74377, 86047)	48628.69 (45888, 52155)
LS Edge (Candidates, k=15)	75448.56 (72416, 81239)	48427.78 (46095, 52408)

Table 1: Comparison of TSPA and TSPB results (Avg, Min, Max)

Method	TSPA Time (avg, min, max) [μs]	TSPB Time (avg, min, max) [μs]
Random Path	0.0226 (0.0209, 0.0361)	0.0234 (0.0196, 0.0482)
LS Edge (Random Path)	106.5689 (94.0115, 159.8687)	107.5312 (92.9847, 132.8901)
LS Edge with LM (Random Path)	50.6919 (35.5602, 69.0622)	53.7797 (39.3278, 92.4221)
LS Edge (Candidates, k=5)	7.1014 (6.4651, 11.1023)	7.5833 (6.8109, 16.3045)
LS Edge (Candidates, k=10)	11.0378 (9.9194, 17.4050)	12.2511 (10.4826, 30.0424)
LS Edge (Candidates, k=15)	15.4697 (13.6906, 21.3452)	17.0768 (15.0540, 24.6822)

Table 2: Average, minimum, and maximum execution times for TSPA and TSPB

5 Conclusion

Based on the results in Tables 1 and 2, the following observations can be made:

The basic Local Steepest Edge algorithm achieves strong solution quality but requires significant computation time due to exhaustive neighborhood evaluation. Introducing the list of moves (LM) mechanism yields nearly identical results, suggesting that computational savings from delta reuse are offset by the overhead of managing and validating saved moves. The frequent invalidation of moves as solutions evolve and the cost of checking edge directions limit the practical benefits of this approach.

The LM mechanism shows theoretical promise for delta reuse but proves less effective when solution structures change dramatically at each iteration. The overhead of maintaining and checking a large list of moves may not justify the limited reusability in highly dynamic search spaces.

6 All Results

Method	TSPA Avg (Min, Max)	TSPB Avg (Min, Max)
Random Path	264188.61 (238257, 293017)	213973.45 (184171, 238914)
Nearest Neighbor (End)	85108.51 (83182, 89433)	54390.43 (52319, 59030)
Nearest Neighbor (Flexible)	73594.10 (71868, 75953)	48694.25 (44609, 57283)
Greedy Cycle	72635.98 (71488, 74410)	51400.60 (49001, 57324)
Greedy 2-Regret	115400.24 (105692, 126860)	72712.85 (67809, 78406)
Greedy 2-Regret (w=0.5)	72136.15 (71108, 73395)	50934.60 (47144, 55700)
NN-Flex 2-Regret	117138.49 (108151, 124921)	74444.46 (69933, 80278)
NN-Flex 2-Regret (w=0.5)	72407.59 (70010, 75452)	47664.46 (44891, 55247)
Local Greedy Node (Greedy Cycle)	70687.00 (70687, 70687)	46443.67 (46328, 51512)
Local Greedy Edge (Greedy Cycle)	70618.84 (70571, 70663)	46089.89 (45824, 51482)
Local Steepest Node (Greedy Cycle)	70687.00 (70687, 70687)	46466.54 (46371, 51512)
Local Steepest Edge (Greedy Cycle)	70571.00 (70571, 70571)	45996.84 (45867, 51259)
Local Greedy Node (Random Path)	85217.85 (77570, 94277)	60178.87 (54006, 66885)
Local Greedy Edge (Random Path)	74232.03 (71777, 77750)	48192.19 (46109, 51934)
Local Steepest Node (Random Path)	85271.00 (85271, 85271)	63491.00 (63491, 63491)
Local Steepest Edge (Random Path)	72687.00 (72687, 72687)	49190.00 (49190, 49190)
Local Steepest (Candidates, k=5)	88563.00 (88563, 88563)	52093.00 (52093, 52093)
Local Steepest (Candidates, k=10)	83598.00 (83598, 83598)	51153.00 (51153, 51153)
Local Steepest (Candidates, k=15)	81136.00 (81136, 81136)	48854.00 (48854, 48854)

Table 3: Comparison of average, minimum, and maximum objective values for TSPA and TSPB. The 3 lowest average values in each column are highlighted in bold.

Method	TSPA Time Avg (Min, Max)	TSPB Time Avg (Min, Max)
Random Path	0.0036 (0.0034, 0.0144)	0.0039 (0.0036, 0.0232)
Nearest Neighbor (End)	0.0694 (0.0463, 0.1093)	0.0685 (0.0470, 0.1282)
Nearest Neighbor (Flexible)	91.0993 (85.8761, 119.1184)	94.7285 (87.4900, 154.2259)
Greedy Cycle	87.6467 (82.1853, 135.8448)	90.6419 (84.6333, 153.7690)
Greedy 2-Regret	13.4090 (12.2387, 18.2542)	13.4612 (12.4574, 21.9434)
Greedy 2-Regret (w=0.5)	12.9063 (11.8320, 18.2641)	12.6926 (11.2819, 20.7204)
NN-Flex 2-Regret	13.8750 (12.7669, 18.4833)	13.9626 (12.9940, 21.1062)
NN-Flex 2-Regret (w=0.5)	13.5540 (12.4458, 19.2731)	13.6345 (12.6257, 21.8523)
Local Greedy Node (Greedy Cycle*)	1.2763 (0.8682, 2.4062)	1.8935 (1.1700, 5.2516)
Local Greedy Edge (Greedy Cycle*)	1.5676 (1.1047, 3.2752)	2.5081 (1.3700, 5.8678)
Local Steepest Node (Greedy Cycle*)	0.5265 (0.4719, 1.0174)	0.7846 (0.4909, 1.6765)
Local Steepest Edge (Greedy Cycle*)	0.5418 (0.4837, 1.0700)	1.0705 (0.6829, 2.2377)
Local Greedy Node (Random Path)	57.7227 (44.0444, 123.7979)	51.7444 (41.5894, 92.8947)
Local Greedy Edge (Random Path)	58.0691 (47.6349, 99.7285)	52.4021 (45.1037, 84.4763)
Local Steepest Node (Random Path)	10.9774 (10.0351, 18.7647)	13.6565 (12.4567, 26.2057)
Local Steepest Edge (Random Path)	9.1549 (8.5878, 12.7278)	9.0840 (8.4419, 18.6255)
Local Steepest (Candidates, k=5)	1.4490 (1.3910, 1.9627)	1.4250 (1.3467, 1.8863)
Local Steepest (Candidates, k=10)	1.8156 (1.7346, 2.3018)	1.7992 (1.7277, 2.4961)
Local Steepest (Candidates, k=15)	2.1838 (2.0927, 2.7651)	2.1374 (2.0609, 2.9937)

Table 4: Comparison of average, minimum, and maximum execution times in microseconds for TSPA and TSPB. Greedy Cycle* is the Greedy Cycle 2-Regret (w=0.5)